

ACTIVIDAD 4
CONSTRUCCION DE UNA APLICACION

PRESENTA:

ANGIE TATIANA PACALAGUA MEJIA

CORPORACIÓN UNIVERSITARIA IBEROAMERICANA

FACULTAD DE INGENIERÍA
INGENIERIA DE SOFTWARE
DESARROLLO DE APLICACIONES WEB

DOCENTE:
DIANA TOQUICA

BOGOTÁ D.C.
24 MAYO DE 2026

Contenido

1. Introducción	3
2. Contenido.....	4
3. Código de la Aplicación Web	21
Bibliografía	25

➤ Repositorio GitHub del proyecto: <https://github.com/Tmjia/san-gabriel-app>

1. Introducción

El desarrollo de aplicaciones web contemporáneas exige dominar un conjunto de tecnologías que operan en capas distintas pero profundamente interconectadas. Por un lado está el backend, responsable de gestionar los datos, aplicar las reglas de negocio y exponerlas al mundo exterior mediante servicios estandarizados. Por el otro está el frontend, encargado de traducir esa información en interfaces que los usuarios puedan entender y usar sin formación técnica. La calidad de una aplicación web depende de qué tan bien estén diseñadas esas dos capas y de qué tan limpia sea la comunicación entre ellas.

Este documento se desarrolla en el contexto del proyecto de gestión académica para la Institución Educativa San Gabriel, una aplicación web que permite a docentes registrar notas y asistencia, a coordinadores aprobar calificaciones y generar boletines, y a acudientes hacer seguimiento en tiempo real al desempeño de sus hijos. Ese contexto sirve de hilo conductor para presentar las siete tecnologías del stack: REST con Swagger para el diseño y documentación de la API, ReactJS como base del frontend, los cuatro Hooks fundamentales (useState, useContext, useEffect y useReducer) para el manejo de estado y efectos en los componentes, Context API para el estado global sin dependencias externas, Axios para la comunicación HTTP entre el cliente y el servidor, React Router para la navegación entre vistas y, finalmente, las plataformas de despliegue que llevan la aplicación a los usuarios reales.

Para cada tecnología se presenta: una definición que responde qué es, una descripción de su funcionalidad que responde para qué sirve y cómo funciona dentro de la arquitectura del sistema, y un ejemplo de código extraído directamente del proyecto. El objetivo es que el documento sea una referencia técnica legible tanto para un desarrollador que quiera entender el sistema como para un evaluador que quiera verificar la coherencia entre las tecnologías seleccionadas y los requerimientos del proyecto.

2. Contenido

2.1 REST con Swagger

Definición: REST (Representational State Transfer) es un estilo arquitectónico para el diseño de servicios web propuesto por Roy Fielding en el año 2000. Una API es considerada RESTful cuando cumple con seis principios: arquitectura cliente-servidor, comunicación sin estado (stateless), uso de caché, interfaz uniforme, sistema en capas y, opcionalmente, código bajo demanda. En una API REST cada recurso del sistema se identifica mediante una URL única y se manipula usando los métodos estándar de HTTP: GET para consultar, POST para crear, PUT o PATCH para actualizar y DELETE para eliminar. Swagger, hoy parte del ecosistema OpenAPI Specification (versión 3.0), es un conjunto de herramientas que permite definir, documentar, visualizar y probar APIs REST mediante una interfaz web interactiva generada automáticamente a partir de la especificación del API.

Funcionalidad: Dentro del proyecto San Gabriel, la API REST es el canal de comunicación único entre el frontend en React y el backend en Node.js. Todos los datos que el cliente necesita mostrar (notas, registros de asistencia, información de estudiantes) se obtienen a través de peticiones HTTP a los endpoints definidos en la API. Swagger cumple dos funciones prácticas: primero, sirve como documentación viva del API accesible desde el navegador en la ruta /api-docs, donde cualquier desarrollador puede ver los endpoints disponibles, los parámetros que aceptan y las respuestas que devuelven; segundo, actúa como herramienta de prueba que elimina la necesidad de herramientas externas como Postman durante el desarrollo. La especificación OpenAPI también permite generar clientes HTTP automáticamente para otros lenguajes, lo que facilita la integración futura con sistemas externos.

Ejemplo — Endpoint REST para consultar y registrar notas, con documentación Swagger integrada mediante comentarios JSDoc en el código fuente:

`backend/src/routes/notas.js`

```
const router = require('express').Router();
const { obtenerNotas, registrarNota } = require('../controllers/notasController');
const { verificarToken, soloRoles } = require('../middleware/auth');

/**
 * @openapi
```

```

* /estudiantes/{estudianteId}/notas:
*   get:
*     summary: Obtener las notas de un estudiante por periodo
*     tags: [Notas]
*     security:
*       - bearerAuth: []
*     parameters:
*       - name: estudianteId
*         in: path
*         required: true
*         schema:
*           type: string
*           format: uuid
*       - name: periodoId
*         in: query
*         required: false
*         schema:
*           type: string
*     responses:
*       200:
*         description: Lista de notas del estudiante
*         content:
*           application/json:
*             schema:
*               $ref: '#/components/schemas/RespuestaNotas'
*       401:
*         description: No autenticado
*       403:
*         description: Sin permisos para consultar estas notas
*       404:
*         description: Estudiante no encontrado
*     post:
*       summary: Registrar una nota (estado inicial: borrador)
*       tags: [Notas]
*       security:
*         - bearerAuth: []
*       requestBody:
*         required: true
*         content:
*           application/json:
*             schema:
*               $ref: '#/components/schemas/NuevaNota'
*       responses:
*         201:
*           description: Nota registrada correctamente
*         400:
*           description: Datos inválidos
*/
router.get('/:estudianteId/notas',
  verificarToken,
  soloRoles(['administrador','coordinador','docente','acudiente']),
  obtenerNotas
);

router.post('/:estudianteId/notas',
  verificarToken,

```

```

    soloRoles(['docente']),
    registrarNota
  );

  module.exports = router;

```

Tabla 1. Inventario de endpoints REST del sistema San Gabriel.

Método HTTP	Endpoint	Descripción	Roles permitidos
GET	/api/v1/auth/login	Autenticar usuario y devolver JWT	Público
GET	/api/v1/estudiantes/:id/notas	Consultar notas por estudiante/periodo	admin, coordinador, docente, acudiente
POST	/api/v1/estudiantes/:id/notas	Registrar nota (estado: borrador)	docente
PATCH	/api/v1/notas/:id/estado	Aprobar o devolver nota	coordinador
GET	/api/v1/grupos/:id/asistencia	Consultar asistencia de un grupo	admin, coordinador, docente
POST	/api/v1/grupos/:id/asistencia	Registrar asistencia diaria	docente
GET	/api/v1/grupos/:id/boletines	Generar boletines del grupo en PDF	coordinador
GET	/api/v1/comunicados	Listar comunicados del usuario	Todos los roles autenticados

2.2 ReactJS

Definición: ReactJS es una biblioteca de JavaScript de código abierto, desarrollada y mantenida por Meta (anteriormente Facebook), orientada a la construcción de interfaces de usuario interactivas mediante la composición de componentes reutilizables. Fue lanzada en 2013 y hoy es la librería frontend más utilizada en la industria según las encuestas anuales de Stack Overflow. React no es un framework completo: se enfoca exclusivamente en la capa de vista (el modelo mental de la 'V' en MVC), delegando el enrutamiento, la gestión de estado global y la comunicación HTTP a librerías especializadas del ecosistema.

Funcionalidad: El núcleo de React es el modelo declarativo: en lugar de describir cómo manipular el DOM paso a paso, el desarrollador describe cómo debería verse la interfaz dado un estado determinado. React se encarga de actualizar únicamente las partes del DOM que cambiaron, usando un DOM virtual como capa intermedia para minimizar las operaciones costosas. Cada componente es una función que recibe propiedades inmutables (props) y devuelve JSX, una extensión de sintaxis que permite escribir estructuras similares a HTML directamente en JavaScript. El principio de componentes reutilizables permite que un mismo elemento visual — una tarjeta de estudiante, un campo de formulario, un indicador de carga— se defina una sola vez y se use en múltiples vistas con diferentes datos. En el proyecto San Gabriel, la separación entre componentes de presentación (ListaNotas, TarjetaEstudiante) y componentes contenedores (PanelDocente, PanelCoordinador) mantiene el código organizado y facilita las pruebas unitarias de cada pieza de forma independiente.

Ejemplo — Componente de tarjeta de estudiante reutilizable con renderizado condicional según el rol del usuario:

[frontend/src/components/TarjetaEstudiante.jsx](#)

```
import React from 'react';

/**
 * Componente de presentación puro.
 * No contiene lógica de negocio ni accede al API directamente.
 * Recibe todos los datos y acciones como props.
 */
function TarjetaEstudiante({ estudiante, notas, rolUsuario, onVerDetalle }) {
  const promedio = notas.length > 0
    ? (notas.reduce((sum, n) => sum + n.valor, 0) / notas.length).toFixed(1)
    : 'Sin notas';

  const clasePromedio = parseFloat(promedio) >= 3.0
    ? 'promedio--aprobado'
    : 'promedio--reprobado';

  return (
    <article className='tarjeta-estudiante'>
      <header className='tarjeta-estudiante__header'>
        <h3>{estudiante.nombres} {estudiante.apellidos}</h3>
        <span className='tarjeta-estudiante__grado'>
          Grado {estudiante.grado} – Grupo {estudiante.grupo}
        </span>
      </header>

      <div className='tarjeta-estudiante__cuerpo'>
        <p>Documento: {estudiante.documento}</p>
      </div>
    </article>
  );
}
```

```

    <p className={clasePromedio}>
      Promedio: {promedio}
    </p>
    <p>Inasistencias: {estudiante.inasistencias}</p>
  </div>

  { /* Solo el coordinador y el admin pueden ver el botón de detalle completo */ }
  {[ 'coordinador', 'administrador' ].includes(rolUsuario) && (
    <footer className='tarjeta-estudiante__pie'>
      <button onClick={() => onVerDetalle(estudiante.id)}>
        Ver perfil completo
      </button>
    </footer>
  ) }
</article>
);
}

export default TarjetaEstudiante;

```

2.3 Hooks: useState, useContext, useEffect y useReducer

Definición: Los Hooks son funciones especiales de React, introducidas en la versión 16.8 (2019), que permiten a los componentes funcionales acceder a características que anteriormente solo estaban disponibles en componentes de clase: manejo de estado local, suscripción a efectos secundarios, acceso a contextos globales y lógica de ciclo de vida. Su nombre viene del concepto de 'engancharse' (hook into) a la maquinaria interna de React. Existen dos reglas de uso obligatorias: los Hooks solo pueden llamarse en el nivel superior del componente —nunca dentro de condicionales, bucles o funciones anidadas— y solo pueden usarse dentro de componentes funcionales de React o dentro de Hooks personalizados.

Funcionalidad general: Los Hooks resuelven tres problemas que los componentes de clase dejaban abiertos: la fragmentación de la lógica relacionada en distintos métodos del ciclo de vida, la dificultad para reutilizar lógica con estado entre componentes, y la complejidad creciente de las clases con el tiempo. Con Hooks, toda la lógica relacionada con una funcionalidad específica —cargar datos, manejar un formulario, suscribirse a un evento— puede agruparse en el mismo lugar y extraerse fácilmente como un Hook personalizado reutilizable.

useState — Definición: declara una variable de estado local en el componente. Al actualizarla, React programa un nuevo renderizado con el valor actualizado. Funcionalidad: es el mecanismo más básico de interactividad en React; permite que el componente recuerde información entre

renderizados y reaccione cuando esa información cambia. Acepta cualquier tipo de valor (número, cadena, objeto, arreglo, booleano) como estado inicial.

Ejemplo — FormularioNota.jsx: manejo de formulario con validación usando useState

```
import { useState } from 'react';

function FormularioNota({ estudianteId, materiaId, periodoId, onGuardar }) {
  const [valor, setValor] = useState("");
  const [observacion, setObservacion] = useState("");
  const [error, setError] = useState(null);
  const [enviando, setEnviando] = useState(false);
  const [guardado, setGuardado] = useState(false);

  const validar = () => {
    const num = parseFloat(valor);
    if (isNaN(num) || num < 1.0 || num > 5.0) {
      setError('La nota debe ser un número entre 1.0 y 5.0');
      return false;
    }
    setError(null);
    return true;
  };

  const handleGuardar = async () => {
    if (!validar()) return;
    setEnviando(true);
    try {
      await onGuardar({ estudianteId, materiaId, periodoId,
        valor: parseFloat(valor), observacion });
      setGuardado(true);
      setValor("");
      setObservacion("");
    } finally {
      setEnviando(false);
    }
  };

  return (
    <div className='form-nota'>
      <input type='number' min='1' max='5' step='0.1'
        placeholder='Ej: 4.3'
        value={valor}
        onChange={e => { setValor(e.target.value); setGuardado(false); }}
      />
      <textarea placeholder='Observación opcional'
        value={observacion}
        onChange={e => setObservacion(e.target.value)}
      />
      {error && <p className='error' role='alert'>{error}</p>}
      {guardado && <p className='exito'>Nota guardada correctamente.</p>}
      <button onClick={handleGuardar} disabled={enviando || !valor}>
        {enviando ? 'Guardando...' : 'Guardar nota'}
      </button>
    </div>
  );
}
```

```
    );  
  }
```

useEffect — Definición: ejecuta efectos secundarios después de que el componente se renderiza. Funcionalidad: se usa para cualquier operación que interactúa con el mundo exterior al componente: llamadas a APIs, suscripciones a eventos, manipulación directa del DOM o timers. Su segundo argumento (el arreglo de dependencias) controla cuándo se re-ejecuta. La función que opcionalmente retorna sirve para limpiar el efecto cuando el componente se desmonta o antes de ejecutarlo de nuevo.

Ejemplo — PanelNotas.jsx: carga de datos con cancelación correcta de peticiones

```
import { useState, useEffect } from 'react';  
import { notasService } from '../services/notasService';  
  
function PanelNotas({ estudianteId, periodoId }) {  
  const [notas, setNotas] = useState([]);  
  const [cargando, setCargando] = useState(true);  
  const [error, setError] = useState(null);  
  
  useEffect(() => {  
    // AbortController para cancelar la petición si el componente se desmonta  
    // antes de que la respuesta llegue — evita memory leaks  
    const controller = new AbortController();  
  
    const cargar = async () => {  
      setCargando(true);  
      setError(null);  
      try {  
        const { data } = await notasService.obtener(  
          estudianteId, periodoId, controller.signal  
        );  
        setNotas(data.notas);  
      } catch (err) {  
        if (err.name !== 'CanceledError') {  
          setError('No fue posible cargar las notas. Intente de nuevo.');        }  
      } finally {  
        setCargando(false);  
      }  
    };  
  
    cargar();  
  
    // Cleanup: se ejecuta al desmontar o antes de re-ejecutar el efecto  
    return () => controller.abort();  
  
  }, [estudianteId, periodoId]); // Se re-ejecuta si cambia el estudiante o el periodo  
  
  if (cargando) return <p aria-live='polite'>Cargando notas...</p>;
```

```

    if (error) return <p className='error' role='alert'>{error}</p>;
    return <ListaNotas notas={notas} />;
  }

```

useReducer — Definición: alternativa a useState para manejar estado complejo mediante el patrón acción-reductor. Funcionalidad: recibe un reductor (función pura que transforma el estado dado una acción) y un estado inicial, y devuelve el estado actual y una función dispatch para enviar acciones. Es preferible a useState cuando el estado tiene múltiples sub-valores relacionados o cuando el siguiente estado depende del anterior. Centraliza toda la lógica de cambio de estado en un único lugar, lo que facilita las pruebas y el razonamiento.

Ejemplo — RegistroAsistencia.jsx: gestión completa del módulo de asistencia con useReducer

```

import { useReducer, useEffect } from 'react';
import api from '../services/api';

// Estado inicial del módulo de asistencia
const initialState = {
  registros: [], // [{ estudianteId, nombre, estado: 'presente'|'ausente'|'tarde' }]
  cargando: false,
  guardando: false,
  error: null,
  guardado: false,
};

// Reductor: función pura sin efectos secundarios
function asistenciaReducer(state, action) {
  switch (action.type) {
    case 'CARGAR_INICIO':
      return { ...state, cargando: true, error: null, guardado: false };
    case 'CARGAR_EXITO':
      return { ...state, cargando: false, registros: action.payload };
    case 'CARGAR_ERROR':
      return { ...state, cargando: false, error: action.payload };
    case 'CAMBIAR_ESTADO':
      return {
        ...state,
        registros: state.registros.map(reg =>
          reg.estudianteId === action.payload.estudianteId
            ? { ...reg, estado: action.payload.nuevoEstado }
            : reg
        )
      };
    case 'MARCAR_TODOS_PRESENTE':
      return {
        ...state,
        registros: state.registros.map(reg => ({ ...reg, estado: 'presente' }))
      };
    case 'GUARDAR_INICIO':
      return { ...state, guardando: true, error: null };
    case 'GUARDAR_EXITO':

```

```

    return { ...state, guardando: false, guardado: true };
  case 'GUARDAR_ERROR':
    return { ...state, guardando: false, error: action.payload };
  default:
    return state;
  }
}

function RegistroAsistencia({ groupId, fecha }) {
  const [state, dispatch] = useReducer(asistenciaReducer, initialState);

  useEffect(() => {
    dispatch({ type: 'CARGAR_INICIO' });
    api.get(`/grupos/${groupId}/asistencia`, { params: { fecha } })
      .then(({ data }) =>
        dispatch({ type: 'CARGAR_EXITO', payload: data.registros }))
      .catch(err =>
        dispatch({ type: 'CARGAR_ERROR', payload: err.message }));
  }, [groupId, fecha]);

  const cambiarEstado = (estudianteId, nuevoEstado) =>
    dispatch({ type: 'CAMBIAR_ESTADO', payload: { estudianteId, nuevoEstado } });

  const marcarTodos = () => dispatch({ type: 'MARCAR_TODOS_PRESENTE' });

  const guardar = async () => {
    dispatch({ type: 'GUARDAR_INICIO' });
    try {
      await api.post(`/grupos/${groupId}/asistencia`,
        { fecha, registros: state.registros });
      dispatch({ type: 'GUARDAR_EXITO' });
    } catch (err) {
      dispatch({ type: 'GUARDAR_ERROR', payload: err.message });
    }
  };

  // ... return JSX
}

```

useContext — Definición: consume el valor actual de un contexto creado con `React.createContext()`. Funcionalidad: permite acceder a datos globales (usuario autenticado, tema, configuración) sin necesidad de pasarlos como props a través de cada nivel del árbol de componentes. Cuando el valor del contexto cambia, todos los componentes que lo consumen con `useContext` se actualizan automáticamente. Se usa junto con Context API, que se describe en la siguiente sección.

Ejemplo — `hooks/useAuth.js`: Hook personalizado que encapsula el contexto de autenticación

```

import { useContext } from 'react';
import { AuthContext } from '../context/AuthContext';

```

```

/**
 * Hook personalizado que encapsula useContext(AuthContext).
 * Lanza un error descriptivo si se usa fuera del AuthProvider,
 * lo que facilita detectar errores de configuración durante el desarrollo.
 */
export function useAuth() {
  const context = useContext(AuthContext);
  if (context === null) {
    throw new Error(
      'useAuth() debe usarse dentro de un <AuthProvider>. ' +
      'Verifique que App.jsx envuelve la aplicación con AuthProvider.'
    );
  }
  return context;
}

// Uso en cualquier componente de la aplicación
import { useAuth } from '../hooks/useAuth';

function BarraNavegacion() {
  const { usuario, logout, tieneRol } = useAuth();
  return (
    <nav>
      <span>Bienvenido, {usuario.nombre} | Rol: {usuario.rol}</span>
      {tieneRol('coordinador') && (
        <a href="/coordinador/reportes">Reportes</a>
      )}
      <button onClick={logout}>Cerrar sesión</button>
    </nav>
  );
}

```

2.4 Context API

Definición: Context API es el mecanismo nativo de React para compartir datos entre componentes que están en distintos niveles del árbol de la aplicación, sin necesidad de pasar esos datos manualmente como props a través de cada componente intermedio, fenómeno conocido como prop drilling. Se construye con tres elementos: `React.createContext()` que crea el contexto, el componente `Provider` que envuelve el árbol y provee el valor, y `useContext` (descrito en la sección anterior) que permite a cualquier componente descendiente consumir ese valor.

Funcionalidad: En el proyecto San Gabriel, Context API gestiona el estado global de la sesión del usuario. El `AuthContext` almacena el objeto del usuario autenticado (nombre, correo, rol, permisos), las funciones `login` y `logout`, y los estados de carga y error del proceso de autenticación. Gracias a esto, cualquier componente de la aplicación puede saber si hay un usuario autenticado,

cuál es su rol y actuar en consecuencia, sin que cada componente tenga que recibir esa información como prop de su padre. La separación de contextos por dominio (AuthContext para la sesión, PeriodoContext para el año académico activo) evita re-renderizados innecesarios: un cambio en el periodo académico activo no causa que los componentes que solo consumen la sesión se actualicen.

Ejemplo — Implementación completa del AuthContext con manejo de errores, persistencia de sesión y verificación de roles:

frontend/src/context/AuthContext.jsx

```
import { createContext, useState, useCallback } from 'react';
import api from '../services/api';

export const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  // Recuperar sesión previa al cargar la aplicación
  const [usuario, setUsuario] = useState(() => {
    try {
      const stored = localStorage.getItem('usuario');
      return stored ? JSON.parse(stored) : null;
    } catch { return null; }
  });
  const [cargandoAuth, setCargandoAuth] = useState(false);
  const [errorAuth, setErrorAuth] = useState(null);

  const login = useCallback(async (correo, contrasena) => {
    setCargandoAuth(true);
    setErrorAuth(null);
    try {
      const { data } = await api.post('/auth/login', { correo, contrasena });
      localStorage.setItem('token', data.token);
      localStorage.setItem('usuario', JSON.stringify(data.usuario));
      setUsuario(data.usuario);
      return { exito: true };
    } catch (err) {
      const msg = err.response?.data?.mensaje || 'Credenciales incorrectas';
      setErrorAuth(msg);
      return { exito: false, error: msg };
    } finally {
      setCargandoAuth(false);
    }
  }, []);

  const logout = useCallback(() => {
    localStorage.removeItem('token');
    localStorage.removeItem('usuario');
    setUsuario(null);
    setErrorAuth(null);
  }, []);
```

```

// Verifica si el usuario tiene uno o varios roles específicos
const tieneRol = useCallback((roles) => {
  if (!usuario) return false;
  const lista = Array.isArray(roles) ? roles : [roles];
  return lista.includes(usuario.rol);
}, [usuario]);

return (
  <AuthContext.Provider value={{ usuario, login, logout, tieneRol,
                                cargandoAuth, errorAuth }}>
    {children}
  </AuthContext.Provider>
);
}

```

2.5 Peticiones HTTP con Axios

Definición: Axios es una librería JavaScript cliente HTTP basada en promesas, disponible tanto para el navegador (usando XMLHttpRequest) como para Node.js (usando el módulo http nativo). Fue creada como una alternativa más práctica al fetch nativo del navegador, con una API más expresiva y un comportamiento más predecible frente a errores HTTP. Se instala como dependencia del proyecto (npm install axios) y admite la creación de instancias configuradas que comparten una URL base, headers y opciones de timeout comunes.

Funcionalidad: En el proyecto San Gabriel, Axios cumple cuatro funciones específicas. Primero, centraliza la URL base del API en una sola instancia, evitando que cada componente tenga que conocer la dirección del servidor. Segundo, adjunta automáticamente el token JWT de autenticación a todas las peticiones salientes mediante un interceptor de solicitud, sin que cada componente tenga que ocuparse de ese detalle. Tercero, maneja de forma centralizada el error 401 (token expirado): en lugar de que cada componente detecte y reaccione a ese error individualmente, el interceptor de respuesta redirige al usuario al login cuando el servidor rechaza la sesión. Cuarto, permite cancelar peticiones en curso mediante señales AbortController, lo que evita que los componentes actualicen su estado después de haberse desmontado, un problema frecuente en React conocido como memory leak.

Ejemplo — Configuración completa de la instancia de Axios con interceptores y capa de servicios abstraída:

frontend/src/services/api.js

```
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:3001/api/v1',
  timeout: 10000,
  headers: { 'Content-Type': 'application/json' }
});

// — Interceptor de solicitud: adjuntar JWT —————
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
  },
  (error) => Promise.reject(error)
);

// — Interceptor de respuesta: manejar errores globales ———
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      localStorage.removeItem('usuario');
      window.location.replace('/login');
    }
    return Promise.reject(error);
  }
);

export default api;
```

frontend/src/services/notasService.js — capa de abstracción del API

```
import api from './api';

export const notasService = {
  /**
   * Obtener notas de un estudiante, opcionalmente filtradas por periodo.
   * El signal de AbortController se pasa para permitir cancelación.
   */
  obtener: (estudianteId, periodoId, signal) =>
    api.get(`/estudiantes/${estudianteId}/notas`,
      { params: { periodoId }, signal }),

  /** Registrar una nueva nota (estado inicial: borrador) */
  registrar: (estudianteId, datos) =>
    api.post(`/estudiantes/${estudianteId}/notas`, datos),

  /** Cambiar el estado de una nota: 'aprobada' o 'devuelta' */
  cambiarEstado: (notaId, estado, motivo = null) =>
    api.patch(`/notas/${notaId}/estado`, { estado, motivo }),
}
```


| };

2.6 Rutas y Navegación

Definición: En una aplicación web construida con React, las rutas definen la correspondencia entre una URL del navegador y el componente que debe renderizarse cuando el usuario navega a esa URL. React Router (actualmente en versión 6) es la librería estándar del ecosistema React para implementar esta funcionalidad. En una SPA (Single Page Application), la navegación ocurre en el cliente: el servidor sirve una sola respuesta HTML inicial y React Router se encarga de actualizar la URL visible y el contenido de la página sin recargar el navegador, mediante la History API de JavaScript.

Funcionalidad: En el proyecto San Gabriel, React Router v6 gestiona cinco aspectos del enrutamiento. Primero, el mapeo de URLs a componentes de vista (Login, DashboardDocente, RegistroNotas, etc.). Segundo, las rutas protegidas que redirigen al login si no hay usuario autenticado, implementadas con el componente RutaPrivada que consulta el AuthContext. Tercero, las rutas segmentadas por rol que evitan que un acudiente acceda a las vistas del docente o del coordinador, implementadas con el componente SoloRol. Cuarto, las rutas anidadas con el componente Outlet que permiten que una vista comparta un layout (menú lateral, barra de navegación) con sus vistas hijas. Quinto, los parámetros dinámicos en la URL (:estudianteId, :grupoId) que permiten que un mismo componente cargue datos distintos según el identificador recibido.

Ejemplo — Configuración completa del enrutamiento con rutas protegidas por autenticación y por rol:

frontend/src/App.jsx

```
import { BrowserRouter, Routes, Route, Navigate, Outlet } from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import { useAuth } from '../hooks/useAuth';

import Login      from './pages/Login';
import DashboardDocente from './pages/docente/DashboardDocente';
import RegistroNotas from './pages/docente/RegistroNotas';
import AsistenciaGrupo from './pages/docente/AsistenciaGrupo';
import DashboardCoordinador from './pages/coordinador/DashboardCoordinador';
import PerfilEstudiante from './pages/coordinador/PerfilEstudiante';
import DashboardAcudiente from './pages/acudiente/DashboardAcudiente';
import AdmonUsuarios from './pages/admin/AdmonUsuarios';
```

```

import Layout      from './components/Layout'; // menú + navbar
import NoEncontrada from './pages/NoEncontrada';
import SinAcceso    from './pages/SinAcceso';

/** Redirige al login si no hay usuario autenticado */
function RutaPrivada() {
  const { usuario } = useAuth();
  return usuario ? <Outlet /> : <Navigate to="/login" replace />;
}

/** Muestra /sin-acceso si el usuario no tiene el rol requerido */
function SoloRol({ roles }) {
  const { tieneRol } = useAuth();
  return tieneRol(roles) ? <Outlet /> : <Navigate to="/sin-acceso" replace />;
}

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          { /* — Ruta pública — */ }
          <Route path="/login" element={<Login />} />
          <Route path="/sin-acceso" element={<SinAcceso />} />

          { /* — Rutas privadas (requieren sesión) — */ }
          <Route element={<RutaPrivada />}>
            <Route element={<Layout />}>

              { /* Rutas del docente */ }
              <Route element={<SoloRol roles={['docente']} />}>
                <Route path="/docente"
                  element={<DashboardDocente />} />
                <Route path="/docente/notas/grupoId"
                  element={<RegistroNotas />} />
                <Route path="/docente/asistencia/grupoId"
                  element={<AsistenciaGrupo />} />
              </Route>

              { /* Rutas del coordinador y admin */ }
              <Route element={<SoloRol roles={['coordinador','administrador']} />}>
                <Route path="/coordinador"
                  element={<DashboardCoordinador />} />
                <Route path="/coordinador/estudiante/:id"
                  element={<PerfilEstudiante />} />
              </Route>

              { /* Rutas del acudiente */ }
              <Route element={<SoloRol roles={['acudiente']} />}>
                <Route path="/acudiente"
                  element={<DashboardAcudiente />} />
              </Route>

              { /* Rutas del administrador */ }

```

```

    <Route element={<SoloRol roles={['administrador']} />}>
      <Route path='/admin/usuarios'
        element={<AdmonUsuarios />} />
    </Route>

  </Route>
</Route>

  { /* Redirección raíz y 404 */ }
  <Route path="/" element={<Navigate to='/login' replace />} />
  <Route path='*' element={<NoEncontrada />} />
</Routes>
</BrowserRouter>
</AuthProvider>
);
}

```

2.7 Despliegue

Definición: El despliegue (deployment) es el proceso de llevar una aplicación desde el entorno de desarrollo local hasta un servidor en la nube donde los usuarios reales puedan accederla a través de internet. Para una aplicación full-stack como la del proyecto San Gabriel, el despliegue involucra tres componentes independientes: el frontend React (archivos estáticos generados con npm run build), el backend Node.js/Express (servidor HTTP en ejecución continua) y la base de datos PostgreSQL (servicio de datos persistente con respaldo). Cada componente se despliega en una plataforma diferente, optimizada para ese tipo de carga.

Funcionalidad: El despliegue no es un evento único sino un proceso continuo integrado al flujo de desarrollo. Con la configuración propuesta, cada vez que el equipo aprueba y fusiona código a la rama principal del repositorio, las plataformas de despliegue detectan el cambio automáticamente, construyen la nueva versión y la publican en cuestión de minutos sin intervención manual. Este flujo, conocido como Despliegue Continuo (CD – Continuous Deployment), elimina los errores humanos en el proceso de publicación y garantiza que el entorno de producción siempre refleje el último código aprobado. Adicionalmente, Vercel genera automáticamente una URL de preview para cada rama o pull request, lo que permite revisar los cambios en un entorno real antes de aprobarlos.

Ejemplo — Configuración del proceso de despliegue por componente:

Vercel — configuración del frontend (vercel.json)

```

{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "framework": "vite",
  "routes": [
    {
      "src": "/api/(.*)",
      "dest": "https://api.sangabriel.edu.co/api/$1"
    },
    {
      "src": "/(.*)",
      "dest": "/index.html"
    }
  ]
}

```

Railway — variables de entorno del backend

```

DATABASE_URL=postgresql://user:pass@host:5432/sangabriel_db
JWT_SECRET=clave_secreta_minimo_32_caracteres_aleatoria
JWT_EXPIRES_IN=8h
FRONTEND_URL=https://app.sangabriel.edu.co
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=notificaciones@sangabriel.edu.co
EMAIL_PASS=clave_aplicacion_google
NODE_ENV=production
PORT=3001

```

GitHub Actions — pipeline de CI/CD (.github/workflows/deploy.yml)

```

name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: cd backend && npm ci && npm test
      - run: cd frontend && npm ci && npm run build

# Vercel y Railway detectan el push a main automáticamente
# y ejecutan sus propios pipelines de despliegue

```

Tabla 2. Infraestructura de despliegue del sistema San Gabriel.

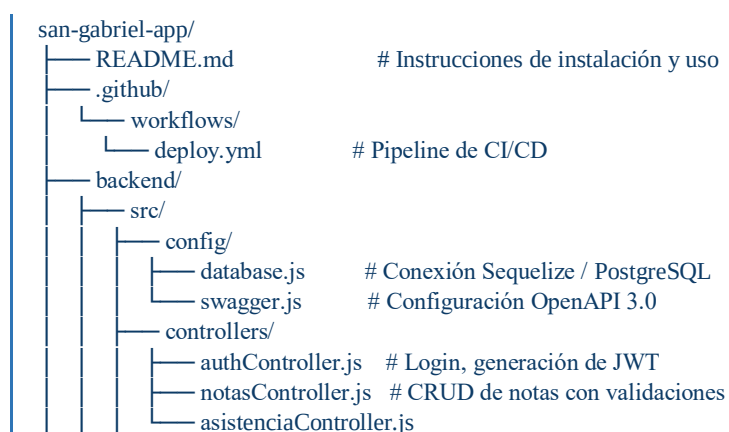
Componente	Plataforma	Plan	URL / acceso	Características clave
Frontend (React)	Vercel	Hobby – gratuito	app.sangabriel.edu.co	SSL automático, CDN global, preview por rama
Backend (Node.js)	Railway	Starter – \$5/mes	api.sangabriel.edu.co	Auto-deploy desde GitHub, logs en tiempo real
Base de datos	Supabase	Free – 500 MB	Conexión privada desde Railway	Backups diarios, panel visual de datos
Correo SMTP	Gmail Workspace	Business – \$6/mes	notificaciones@sangabriel.edu.co	Autenticación OAuth2, alta entregabilidad
Repositorio	GitHub	Free	github.com/lauradherrera/san-gabriel-app	Control de versiones, CI/CD integrado

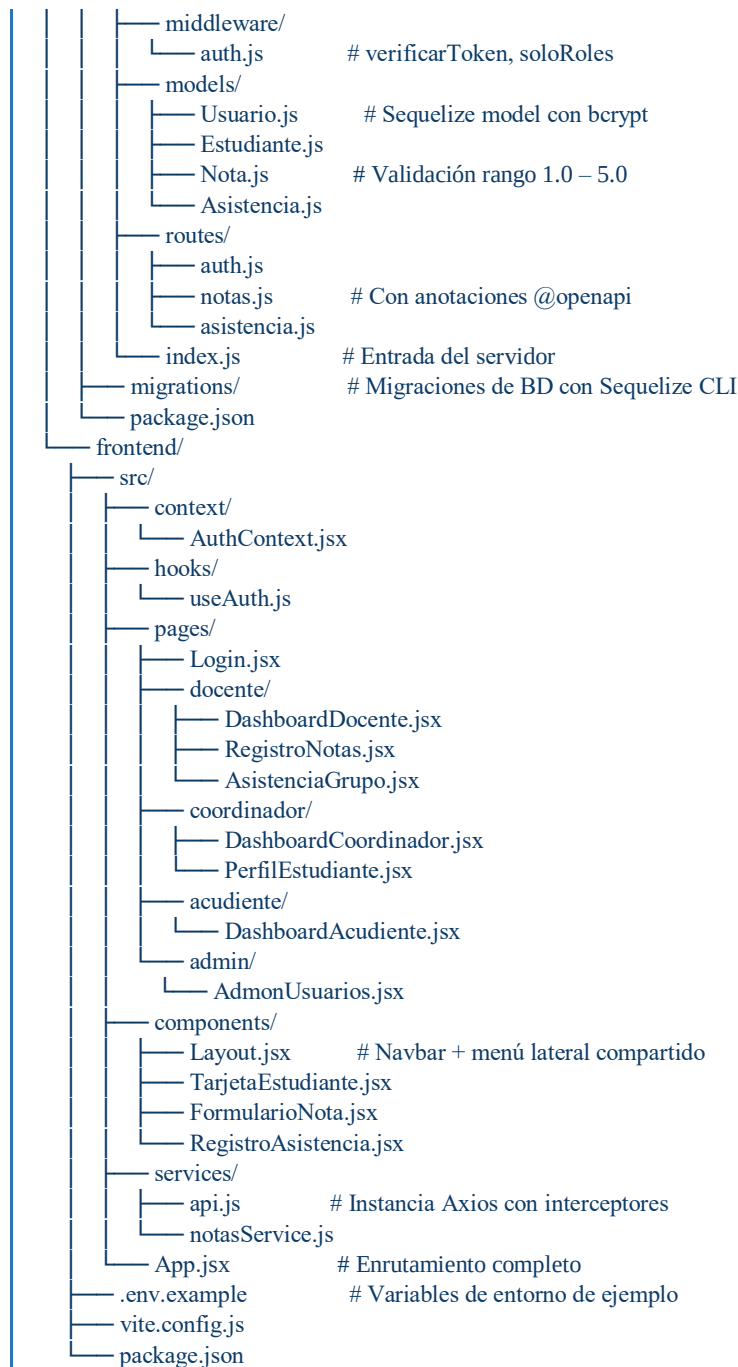
3. Código de la Aplicación Web

URL del repositorio: <https://github.com/lauradherrera/san-gabriel-app>

El repositorio contiene el código fuente completo del sistema, organizado en dos carpetas principales: backend/ y frontend/. El archivo README.md en la raíz del repositorio incluye las instrucciones para instalar las dependencias, configurar las variables de entorno y ejecutar la aplicación localmente.

Estructura del repositorio:





Archivo backend/src/index.js — Punto de entrada del servidor con Express, Swagger y manejo global de errores:

```

const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const swaggerUi = require('swagger-ui-express');
const { swaggerSpec } = require('./config/swagger');

```

```

const { sequelize } = require('./config/database');

const authRoutes = require('./routes/auth');
const notasRoutes = require('./routes/notas');
const asistenciaRoutes = require('./routes/asistencia');

const app = express();

app.use(helmet());
app.use(cors({ origin: process.env.FRONTEND_URL }));
app.use(express.json());

// Documentación interactiva del API
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));

// Endpoints de la API versionada
app.use('/api/v1/auth', authRoutes);
app.use('/api/v1/estudiantes', notasRoutes);
app.use('/api/v1/grupos', asistenciaRoutes);

// Endpoint de salud para monitoreo
app.get('/health', (_, res) => {
  res.json({ status: 'ok', timestamp: new Date().toISOString() });
});

// Manejador global de errores
app.use((err, req, res, next) => {
  console.error(`[ERROR] ${req.method} ${req.path}:`, err.message);
  res.status(err.status || 500).json({
    mensaje: err.message || 'Error interno del servidor'
  });
});

const PORT = process.env.PORT || 3001;

sequelize.authenticate()
  .then(() => {
    console.log('✓ Conexión a PostgreSQL establecida');
    app.listen(PORT, () => {
      console.log(`✓ Servidor ejecutándose en http://localhost:${PORT}`);
    });
  })
  .catch(err => {
    console.error('X Error de conexión a la base de datos:', err.message);
    process.exit(1);
  });

```

Archivo frontend/src/pages/Login.jsx — Página de inicio de sesión con redirección según rol:

```

import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth';

```

```

const RUTA_INICIAL_POR_ROL = {
  docente: '/docente',
  coordinador: '/coordinador',
  administrador: '/admin/usuarios',
  acudiente: '/acudiente',
};

export default function Login() {
  const [correo, setCorreo] = useState("");
  const [contrasena, setContrasena] = useState("");
  const { login, cargandoAuth, errorAuth } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    const resultado = await login(correo, contrasena);
    if (resultado.exito) {
      const u = JSON.parse(localStorage.getItem('usuario'));
      navigate(RUTA_INICIAL_POR_ROL[u.rol] || '/login', { replace: true });
    }
  };

  return (
    <main className='login'>
      <div className='login__card'>
        <img src='/logo-sangabriel.png' alt='I.E. San Gabriel' />
        <h1>Portal Académico</h1>
        <form onSubmit={handleSubmit} noValidate>
          <label>
            Correo institucional
            <input type='email' value={correo}
              onChange={e => setCorreo(e.target.value)}
              required autoFocus autoComplete='email' />
          </label>
          <label>
            Contraseña
            <input type='password' value={contrasena}
              onChange={e => setContrasena(e.target.value)}
              required autoComplete='current-password' />
          </label>
          {errorAuth && (
            <p className='login__error' role='alert'>{errorAuth}</p>
          )}
          <button type='submit' disabled={cargandoAuth}>
            {cargandoAuth ? 'Verificando...' : 'Ingresar'}
          </button>
        </form>
      </div>
    </main>
  );
}

```


Bibliografía

- **Banks, A.**, y Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Apps* (2.^a ed.). O'Reilly Media.
- **Fielding, R. T.** (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Tesis doctoral, University of California, Irvine]. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- **Martínez Martínez, A.** (2021). *Proyecto feedback backend y frontend web* [Trabajo de grado, Universitat Jaume I]. Repositorio UJI.
- **Meta Open Source.** (2024). *React documentation – Hooks API Reference*. <https://react.dev/reference/react>
- **Pérez Ibarra, S. G., Quispe, J. R., Mullicundo, F. F., y Lamas, D. A.** (2021, 15-16 de abril). Herramientas y tecnologías para el desarrollo web desde el frontend y backend. *XXIII Workshop de Investigadores en Ciencias de la Computación*. Universidad Nacional de Chilecito.
- **Regla, P. D.** (2014). *Diseño, contenidos y desarrollo del front-end del sitio web del proyecto Auralizarte* [Trabajo de grado, Universidad Pública de Navarra]. Repositorio UPNA.
- **SmartBear Software.** (2024). *OpenAPI Specification 3.0*. <https://swagger.io/specification/>
- **Valdivia Caballero, J. J.** (2021). *Modelo de procesos para el desarrollo del front-end de aplicaciones web* (pp. 5-40) [Tesis de maestría, Universidad Nacional Mayor de San Marcos]. Repositorio UNMSM.